

Analyse Statique avec PSScriptAnalyzer

Objectif

Mettre en pratique le concept de "Shift-Left". Vous allez auditer un script fonctionnel mais truffé de mauvaises pratiques historiques, analyser le rapport du Linter, et appliquer la remédiation pour atteindre le standard "Zéro Défaut" exigé par une pipeline CI/CD.

Exercice 1 : Préparation du script "Sale" (The Dirty Script)

1. Dans votre dossier C:\Audit\, créez un nouveau fichier nommé Check-Systeme.ps1.
2. Copiez-collez exactement le code suivant (avec ses défauts) et sauvegardez le fichier :

```
function Check-Systeme {  
    param(  
        $Serveur = "localhost"  
    )  
  
    $MotDePasseAdmin = "P@ssw0rd_2026!"  
  
    Write-Host "Lancement de l'audit sur $Serveur..." -ForegroundColor Yellow  
  
    # Récupération des services  
    $services = gwmi Win32_Service -ComputerName $Serveur  
  
    # Filtrage des services en cours d'exécution  
    $actifs = $services | ? { $_.State -eq 'Running' }  
  
    echo "Nombre de services actifs : $($actifs.Count)"  
}
```

Remarque : Si vous lancez ce script, il fonctionne. C'est tout le problème de l'administration classique : le code s'exécute, donc l'administrateur pense qu'il est bon.

Exercice 2 : L'Audit Impitoyable

Vous allez maintenant endosser le rôle de la pipeline d'intégration continue.

1. Dans votre console, lancez l'audit statique sur votre fichier :
Invoke-ScriptAnalyzer -Path "C:\Audit\Check-Systeme.ps1"

2. Analyse du rapport :

- Combien de violations (lignes) le moteur vous remonte-t-il ?
- Repérez la violation de sévérité **Error** concernant la sécurité. Quelle est la règle déclenchée ?
- Repérez les violations concernant la syntaxe (gwmi, ?, echo). Quelle est la règle déclenchée ?
- Quelle règle reproche au script le nom même de sa fonction ?

Exercice 3 : La Remédiation (Refactoring)

Votre objectif est de corriger le fichier Check-Systeme.ps1 pour que Invoke-ScriptAnalyzer ne renvoie plus **aucune ligne**.

Aide pour la remédiation :

1. **Nom de la fonction** : Remplacez le verbe Check par un verbe approuvé (ex: Test ou Get).
2. **Mot de passe** : Supprimez cette variable en texte brut. (En production, on utiliserait un paramètre [PSCredential] ou le module SecretManagement).
3. **Write-Host** : Remplacez-le par Write-Verbose ou Write-Information.
4. **Les Alias** :
 - Remplacez gwmi par Get-WmiObject (ou mieux, Get-CimInstance).
 - Remplacez ? par Where-Object.
 - Remplacez echo par Write-Output.

Validation :

Relancez Invoke-ScriptAnalyzer -Path "C:\Audit\Check-Systeme.ps1".

Le silence de la console (aucun retour) est la preuve absolue que votre code est certifié pour la production.

Exercice 4 (Bonus DevOps) : Le Fichier de Tolérance

Parfois, on écrit de petits outils graphiques où l'on a vraiment besoin de Write-Host pour faire des menus colorés, et on ne veut pas que la CI/CD échoue pour cela.

1. Créez un fichier de paramètres nommé C:\Audit\LinterSettings.psd1 contenant ceci :

```
@{
    IncludeDefaultRules = $true
    ExcludeRules = @("PSAvoidUsingWriteHost")
}
```
2. Remettez volontairement un Write-Host dans votre script PowerShell.
3. Lancez l'analyse en appliquant votre fichier de configuration :

```
Invoke-ScriptAnalyzer -Path "C:\Audit\Check-Systeme.ps1" -Settings
"C:\Audit\LinterSettings.psd1"
```

Constat : L'analyseur valide votre code car vous avez documenté et assumé formellement cette exception technique.